

Multilevel Frequent Pattern Mining on Pipeline Incident Data

Brandon Narine
University of Manitoba
Winnipeg, Canada
narineb@myumanitoba.ca

Felix Wedel
University of Manitoba
Winnipeg, Canada
wedelf@myumanitoba.ca

JY Li
University of Manitoba
Winnipeg, Canada
lij31@myumanitoba.ca

Pipeline incidents occur throughout the world and can have devastating financial and ecological impacts. A large amount of data is collected and made publicly available for each pipeline incident. While some important information is explicitly visible in such datasets, a lot of implicit information remains hidden. Frequent pattern mining approaches such as multi-level mining may provide access to some of this implicit data that was previously hidden. Other techniques, such as natural language processing and neural networks have already been applied in order to find causal factors in pipeline incidents which can be used for prevention. By applying multi-level mining to pipeline incident data, association rules between data columns in the pipeline incident dataset can be formed and used to gain new insights into pipeline incidents that may help improve the safety and reliability of pipelines.

Keywords—Multi-level mining, FPM, association rules, pipeline, incidents

I. INTRODUCTION

In the year of 2021 alone, 99 Canadian companies utilized approximately 69,400 kilometers of government regulated pipelines to transport natural gas, oil and other substances [14]. While often considered as a safer alternative to vehicular transport, pipelines are still susceptible to accidents and failures. Due to the nature of the substances that are transported through these pipelines, these incidents can often be extremely costly both financially and environmentally [7]. While relatively rare, some pipeline incidents result in serious injuries and sometimes fatalities. Research and development of ways to mitigate or prevent these incidents has been an ongoing endeavor, however pipeline

incidents remain a constant threat with the average number of annual pipeline incidents in Canada from 2011 to 2021 sitting at 117 [14]. Recent Data also seems to suggest that the number of annual pipeline incidents in Canada has been relatively stable within the last couple of years [14]. With the demand for energy increasing, it seems likely that we will also see an increase in the construction and use of natural gas and oil pipelines in the near future. Because of this, it is important to find ways to prevent pipeline incidents from happening and to find ways to mitigate the negative impacts of pipeline incidents.



Pipeline incidents plotted on a map of Canada.

Many organizations, including government organizations such as the Canada Energy Regulator (CER) or formerly National Energy Board (NEB), collect data related to pipeline incidents that occur within their country [14]. This data is often publicly available and contains detailed information about each incident. This includes information like severity, causes of the incident and some maintenance history details [11]. This historical data contains a lot of useful information that could potentially be used to prevent these incidents or predict when they occur.

As is often the case with large and complex datasets, not all of the useful information is explicitly listed, nor can it be easily extracted through conventional means. This becomes increasingly apparent for datasets such as the pipeline incident dataset compiled and managed by CER, which contains data columns with implicit hierarchies that may hide certain features and relationships between the data [11]. At present, CER's pipeline incident dataset contains records for 1736 pipeline incidents that have occurred across Canada since the year 2008 [11]. Each record contains 102 different columns of data [11]. This data ranges from dates to text descriptions and various counts. While this data lends itself well to many statistical models and machine learning techniques, such as linear regression modeling and k-means clustering, our work involves extracting interesting relationships and patterns from the dataset and its implicit data hierarchies using data mining techniques that stem from frequent pattern mining.

Frequent pattern mining involves extracting relationships between different items in a database entry that occur often enough to be considered frequent [2]. From these relationships, or frequent patterns, we can generate association rules that allow us to make predictions about future events and to predict the likelihood of sets of items being present in a database entry, given the presence of another item [2]. Frequent pattern mining alone could be used to obtain many interesting frequent patterns and association rules from pipeline incident data. This data could then be used for predictions and classifications related to pipeline incidents. As previously mentioned however, CER's pipeline incident data includes natural data hierarchies embedded within certain data columns. Using a basic frequent pattern mining approach would fail to uncover certain patterns within the different levels of these hierarchies. In order to uncover interesting frequent patterns and association rules at the different data-hierarchy levels, other techniques such as multi-level mining must be used. Multi-level frequent pattern mining allows us to uncover associations between different variables that have natural data hierarchies, such as spatial and temporal data. This can be applied to the pipeline incident data to discover patterns between the different specificity levels contained within a single date or location related column [13]. For example, by using multi-level mining, we can uncover patterns related not

only to a specific year, month, day and combination, but also between years and months separately.

The goal of our work is to provide useful insights into pipeline incidents and the events that lead up to them. By applying the technique of multi-level mining to pipeline incident data, we hope to uncover previously unknown or poorly understood relationships embedded within the data. These insights have the potential to help with predictions of when and where pipeline incidents may occur, as well as the severity and cause of the incidents. With these predictions, new informed decisions can be made about incident prevention, new safety measures and damage control measures. Our hope is that these new insights will help reduce the frequency and impact of pipeline incidents and perhaps prevent future injuries or fatalities from occurring.

II. BACKGROUND AND EXISTING WORK

A. Frequent pattern and Multi-level mining

Much of the foundation for our frequent pattern and multi-level mining approaches come from two influential data-mining papers [2][13]. Aggarwal et al. [2] have done a tremendous job of compiling a plethora of techniques, concepts and algorithms for various types of frequent pattern mining in their book on frequent pattern mining. The work done by T. Eavis and X. Zheng [13] takes some of the ideas outlined in the work by Aggarwal et al. [2] in order to present both naive approaches to multi-level mining as well as novel algorithms that are specialized and efficient at multi-level mining. Our work makes use of some of these techniques and algorithms, such as the Apriori algorithm and iterative multi-level mining, and applies them to pipeline incident data in order to extract useful information [2][13].

B. Pipeline Incident Prevention

Other research has gone into understanding and preventing pipeline accidents using methods other than multi-level mining. For example, the concepts of natural language processing and text mining have gone towards understanding pipeline incidents and what causes them [5]. This approach focuses only on the text data collected and is a fundamentally different approach from multi-level mining.

C. Causal Factors of Pipeline Incidents

Some research has also gone into improving the understanding of pipeline incident datasets and

analyzing the causal factors listed in the incident reports using neural networks [10]. This research could serve to improve data collection for pipeline incidents which may result in better results when applying techniques like multi-level mining or natural language processing to these datasets.

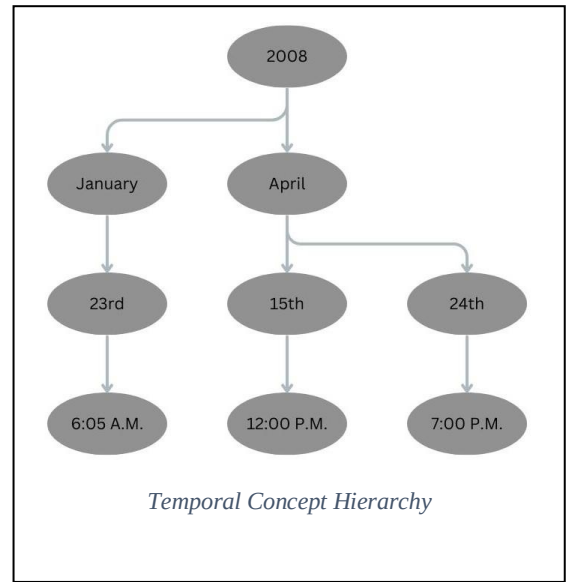
D. Impacts of Pipeline Incidents

Research into the negative effects of pipeline incidents such as the work done by Aroh et al. [7] is also an important contributing factor to this project. Not only does this research provide a better understanding of the impacts that pipeline incidents may have, but it also provides motivation to work towards solutions and preventative measures. Without research into the consequences of pipeline incidents, this project and similar projects may have never been undertaken.

III. DATA PREPROCESSING

In order to extract relationships and frequent patterns from the different hierarchies embedded within the spatial and temporal data columns within CER's pipeline incident dataset, the hierarchies first need to be separated into separate columns. This allows us to include only the levels of the hierarchy that we wish to mine at a given time, using the naïve approach to multi-level mining [13]. The spatial and temporal hierarchies within the data set are distinct from each other, containing different levels and data types, and therefore require different processes to separate.

A. Temporal Data



Dealing with temporal data is a relatively simple matter. The temporal data in CER's pipeline incident dataset is all in the form of simple date strings. One inconsistency between columns containing temporal data is that some columns, namely the columns relating to when the incident was discovered and when the incident occurred, contain information as specific as the hour. Other columns containing temporal data only have information as specific as the day. Another inconsistency is that some date columns contain dates in the form of MM/DD/YYYY, whereas other columns have dates in the form of YYYY-MM-DD. Despite these inconsistencies, each date column could be easily separated into multiple columns, where each column contains one piece of the date. This was done using a python script that performed the following steps:

- Read the dataset from a CSV file into a data frame.
- Extract the date columns and store them in separate one column data frames
- For each date column create a new one-column data frame for each piece of information contained in the date string, then read each row and:
 - Split the date string on either '/' or '-' characters

- Insert each resulting string after the split into one of the new data frames

For a date column containing year, month, day and hour, this results in four new one-column data frames, each representing one piece of the date/time as shown in the figure below.

2007-12-27 14:00	2007	12	14:0
2008-01-02 11:30	2008	1	11:30
2008-01-23 6:05	2008	1	6:5
2008-01-26 10:45	2008	1	10:45
2008-01-27 0:01	2008	1	0:1
2008-02-22 0:00	2008	2	0:0
2008-02-25 0:00	2008	2	0:0
2008-03-02 10:30	2008	3	10:30
2008-03-01 16:45	2008	3	16:45
2008-03-14 13:30	2008	3	13:30
2008-03-24 9:50	2008	3	9:50
NA	NA	NA	NA
2008-04-16 14:30	2008	4	14:30
2008-04-21 7:00	2008	4	7:0

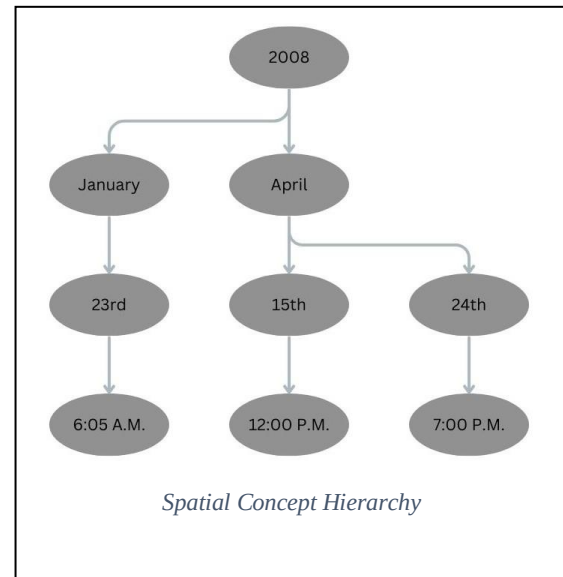
These data frames could then be exported as .csv files and loaded using Microsoft Excel. The original date format already represents the first level of our data hierarchy, but to represent the second, third and, in the case that the column contains an hour, the fourth level of the hierarchy, we use excel to combine certain columns. For date columns as specific as hour, the columns representing the data hierarchies look as follows.

2007-12-27 14:00	2007	Dec-07	2007-12-27
2008-01-02 11:30	2008	Jan-08	2008-01-02
2008-01-23 6:05	2008	Jan-08	2008-01-23
2008-01-26 10:45	2008	Jan-08	2008-01-26
2008-01-27 0:01	2008	Jan-08	2008-01-27
2008-02-22 0:00	2008	Feb-08	2008-02-22
2008-02-25 0:00	2008	Feb-08	2008-02-25
2008-03-02 10:30	2008	Mar-08	2008-03-02
2008-03-01 16:45	2008	Mar-08	2008-03-01
2008-03-14 13:30	2008	Mar-08	2008-03-14
2008-03-24 9:50	2008	Mar-08	2008-03-24
NA	NA	NA	NA
2008-04-16 14:30	2008	Apr-08	2008-04-16
2008-04-21 7:00	2008	Apr-08	2008-04-21

Apart from data pertaining to when the incident occurred and when it was discovered, the time between these events also seemed potentially interesting. CER's pipeline incident dataset does not explicitly contain this information. Using the

occurrence and discovery information however, a value representing the time between the two was easily obtained. This was done by converting the date/time in the occurrence and discovery columns into the POSIXct date format which stores time as number of seconds since 1970. This allows the use of simple arithmetic on the resulting date values to get a floating point value representing the hours between the two events.

B. Spatial Data



While CER's pipeline incident dataset already contained two separate levels of spatial data, we decided to make use of a reverse geocoding API in order to convert latitude and longitude values into addresses. By using a python script to scan through the latitude and longitude values and individually send them to the *HERE* reverse geocoding API, a list of addresses pertaining to the pipeline incidents could be compiled using the json objects returned by the API. For each latitude and longitude pair, the json object returned by the API was parsed to extract the relevant information so that it could be stored in a data frame and then exported as a .csv file. Initially, the plan was to use the street value returned by the API, but since pipelines often run through unpopulated and undeveloped areas, a large number of coordinates resulted in NA values for the nearest street so the decision was made to exclude this information. While very few of the coordinate pairs resulted in NA values for the city portion of the address, many of these values referred to

municipalities or districts within a given province. This was again likely due to the fact that pipelines do not only run through populated areas. The resulting .csv file was then loaded into Microsoft Excel and the few missing values were manually filled in by searching the coordinate pairs in Google Maps to get the missing addresses. The municipality strings were

province	municipality	coordinates
Alberta	Greenview_No_16, Alberta	54.84, -118.65
Manitoba	Pipestone, Manitoba	49.753, -101.239
Manitoba	Pipestone, Manitoba	49.754, -101.237
British_Columbia	Northern_Rockies, British_Columbia	58.012, -122.694
British_Columbia	Peace_River, British_Columbia	56.143, -120.516
Alberta	Beiseker, Alberta	51.385, -113.535
Saskatchewan	Moosomin_No_121, Saskatchewan	50.207, -101.487
Ontario	Dryden, Ontario	49.789, -92.765
Ontario	Fauquier-Strickland, Ontario	49.303, -82.043
Manitoba	Oakview_RM, Manitoba	50.076, -99.998
Saskatchewan	Progress_No_351, Saskatchewan	51.92, -109.17
Ontario	Timiskaming, Ontario	47.446, -79.762
Ontario	Kenora_District, Ontario	49.808, -94.189
Saskatchewan	Regina, Saskatchewan	50.48, -104.566
British_Columbia	Peace_River, British_Columbia	56.279, -120.963
British_Columbia	Peace_River, British_Columbia	55.338, -121.871
Manitoba	Pipestone, Manitoba	49.754, -101.239

then combined into one column with the corresponding province, which was already present in the dataset. This was done so that two entries could be distinguished in the case that two provinces happened to have a municipality or city of the same name. The three resulting levels of the spatial data hierarchy are province, municipality and coordinates, as can be seen in the figure below.

After extracting the data hierarchies from the spatial and temporal data columns, original versions of these columns in the pipeline incident data set were replaced by a column representing a specific level of the hierarchy. Since the occurrence date and discovery date columns had four hierarchy levels, the end result is four different datasets. Each of these data sets contains spatial and temporal data at a specific level of their respective data hierarchies. For columns only containing three levels, the same values are carried over from the third to fourth level data set. Columns that contain comma-separated values, such as the “detailed what happened” column can contain a list of descriptions, were also separated into multiple columns in preparation for encoding. This was done so that each piece of the comma-separated value could be mined individually. For example, if the original value was “cracking, material loss,” these would be separated so that “cracking” and “material

loss” showed up separately in the data set entry. This allows the mining algorithm to count the values individually rather than counting the combination of the two values. In other words, if two columns originally contained “cracking, material loss” and “cracking, corrosion,” after being separated, the mining algorithm would be able detect the “cracking” value in both entries, which was not the case before separation. Another problem with keeping the comma-separated values in a single column is that entries containing the same set of values in different orders would have also been counted separately, which is not desirable. The last step before encoding the data set was to get rid of unnecessary mostly empty columns. Unique data columns, such as the pipeline incident number column, were removed since they do not provide any real insight when performing frequent data mining and they would increase the amount of columns in the encoded data set by the number of entries in the column. Columns that contained over 50 percent empty values were also dropped in order to focus on columns that were more likely to produce frequent patterns.

C. One-Hot Encoding

The final step of data preprocessing is to one-hot encode each of the datasets that will be mined. The first step of the one-hot encoding is to create dummy variables for each of the possible values in a given column. For a column with k possible values, this results in a total of k dummy variables that are each represented as columns in the one-hot encoded dataset. Each entry in these new columns is a binary value that represents whether or not that dummy variable was present in the original data entry. This allows the frequent pattern mining algorithm to look at binary values rather than having to look at and compare the various data types contained in each column of the original data set and only having to look at the string name of the dummy variable if it reads in a value of one for that entry. For the purposes of frequent pattern mining, one-hot encoding also takes care of null or missing values since they will simply contain 0’s for each of the dummy variables generated from the original column, and will therefore be ignored by the algorithm. While one-hot encoding vastly increases the number of columns in the dataset, it also allows all entry values to be stored as simple binary values and makes frequent pattern mining much simpler and more efficient in the long run.

IV. GENERATING ASSOCIATION RULES

A. Multi-level Frequent Pattern Mining

Since the CER pipeline incident dataset is not a massive dataset, mining frequent patterns from it does not require a large amount of computational power. Because of this, an implementation of the Apriori frequent pattern mining algorithm was used to mine the pipeline incidents dataset. The Apriori algorithm was chosen over other algorithms like FP-growth simply because it was extremely easy to find existing python libraries such as mlexend, that had well implemented versions of the algorithm that were very simple to use and were designed for one-hot encoded data. Since the frequent patterns were needed immediately, there was also no need for the delayed mining property of algorithms like FP-growth. Despite Apriori being generally less efficient than algorithms like FP-growth, all versions of the preprocessed and one-hot encoded pipeline incident dataset could be mined in less than a second, even with a minimum support value of 50%, which was more than good enough for the purposes of this project. Attempting to mine at lower minimum support values initially caused some memory issues because the binary values in each column of the dataset were being stored in memory as 64-bit integers. This issue was solved by explicitly reading them in as 8-bit integers, which greatly reduced the memory required.

B. Association Rules

Generation of association rules was also done using an algorithm imported from the mlexend python library. This algorithm would return the antecedents and consequents of each association rule separately in the form of frozensets, as well as a number of metrics to evaluate and filter out rules. Along with the support of the consequents, antecedent and both combined, each association rule is returned with its confidence, lift, leverage, conviction and zhangs metric. These metrics are important when filtering out interesting and useful association rules, especially when there could potentially be thousands of rules generated.

In combination with the encoded datasets for each level of the data hierarchies within the pipeline incident dataset, the Apriori and association rule algorithms could be used to mine each dataset individually to obtain frequent patterns and

association rules. Based on the hierarchy level of the dataset, the minimum support of frequent patterns

Data Hierarchy Level	Minimum Support	Number of Frequent Patterns	Minimum confidence	Minimum lift	Minimum conviction	Number of Association rules
Level 1	0.85	483	99%	>1	>4	465
Level 2	0.75	2845	99%	>1	>4	2022
Level 3	0.65	9247	99%	>1	>10	10519
Level 4	0.55	34137	99%	>1.5	>15	27270

could be adjusted, since patterns at lower levels of the hierarchy are much less likely to appear because of their specificity.

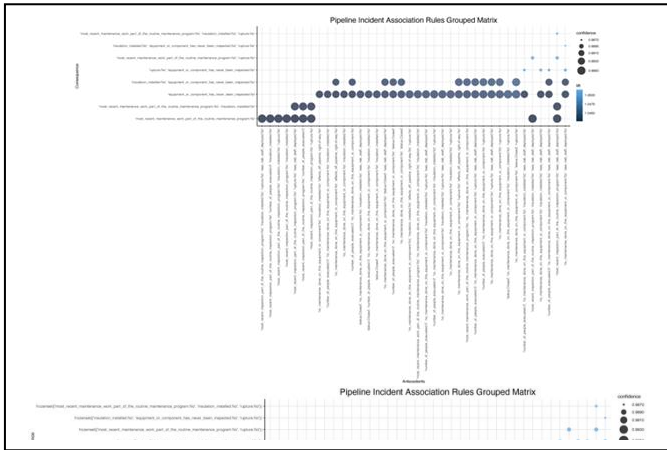
After some experimentation, the following values were settled upon for the final run of frequent pattern mining and association rule generation.

C. Visualizing Association rules

To visualize the association rules, we created a grouped matrix using a balloon plot. The balloon plot is a scatter plot with the antecedent and consequence on the x and y axes respectively. Each data point in the plot represents an antecedent and consequence pair, and the size of the balloon corresponds to the confidence level of the pair. Larger balloons indicate a higher confidence level. In addition to size, the color of the balloons represents the lift of the association rule. Lift is a measure of how much more often the antecedent and consequence occur together compared to what would be expected by chance. We used a shade of blue for the balloon color, with darker shades indicating higher lift values [8].

To highlight the most important associations, we reordered the antecedents and consequences by lift value. This ensured that the association rules with the highest lift values were plotted in the top right corner of the plot. By doing so, we could quickly identify the most significant associations in the dataset.

The following figure shows a sample output of the visualization. This data was generated by selecting the top 15 association rules with the highest lift values on the first level of the analysis.



used to make it easier to identify the column variable associated with each value in the association rules."

Since ruptures in the dataset were a rare occurrence, it makes sense that the "No" value for that variable is frequent, therefore we can ignore that part of the consequence.

Furthermore, there is a strong relationship between pipeline incidents where the pipeline was not inspected recently as part of the routine inspection program, no NEB staff were deployed, no recent maintenance work was done as part of the routine maintenance program, and no insulation was installed. This could be a useful insight for pipeline maintenance and safety practices, as it suggests that these variables may be interrelated and could be worth considering as a group when identifying potential risk factors.

Visualizations of the association rules derived from all four dataset sets can be seen below.

V. CONCLUSION

A. Results

The most interesting association rule in our analysis was determined based on the highest lift value. This association rule has the following antecedent and consequence:

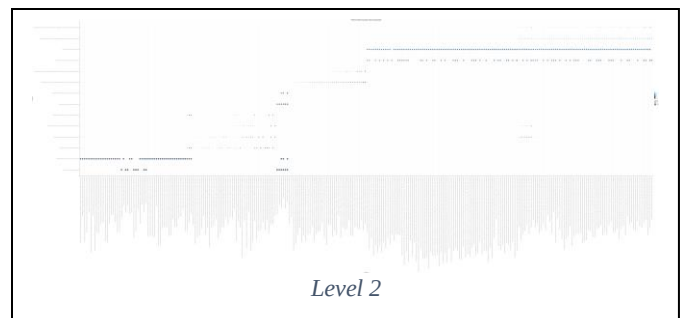
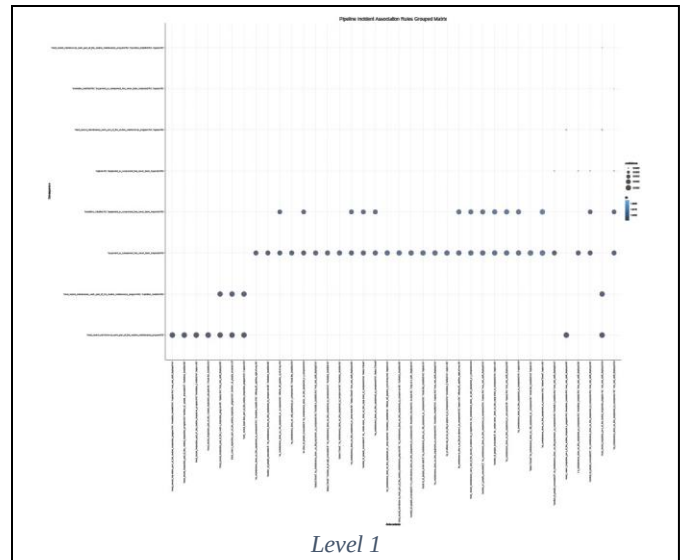
Antecedent:

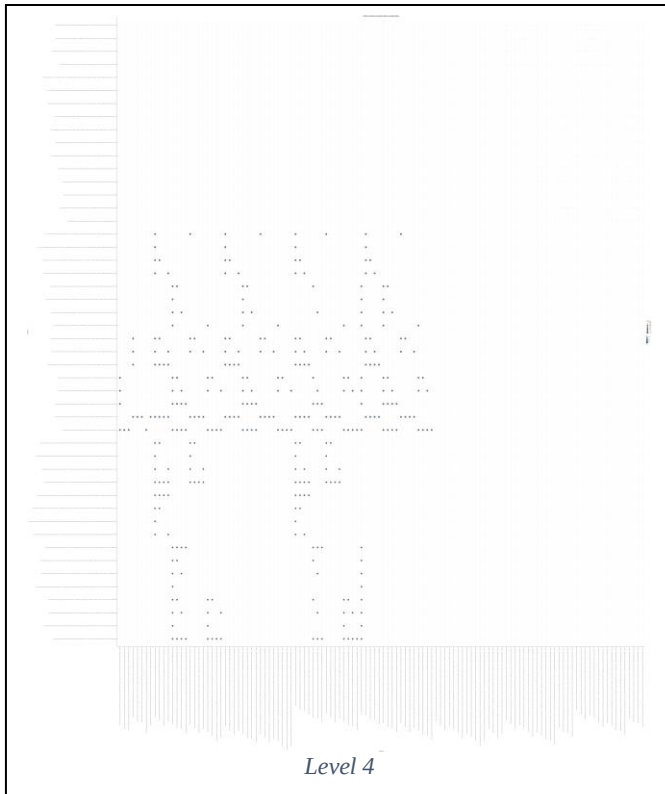
{'most_recent_inspection_part_of_the_routine_inspection_program.No', 'was_neb_staff_deployed.No'})

Consequent:

{'most_recent_maintenance_work_part_of_the_routine_maintenance_program.No', 'insulation_installed.No', 'rupture.No'})

The text after the "." in each item represents the value of the column variable which is before the ".". For example, 'most_recent_inspection_part_of_the_routine_inspection_program.No' refers to the column variable 'most_recent_inspection_part_of_the_routine_inspection_program' and its value 'No'. This convention was





B. Limitations

Originally, the plan was to implement a version of the FP-Growth algorithm that worked well for our dataset so that the algorithm could be easily extended or changed to meet future needs. During implementation, however, a number of issues came

up. Understanding the process of creating an FP-tree can be difficult because it involves multiple steps, such as sorting items by frequency, building a header table, and recursively adding transactions to the tree. It is challenging to debug the tree construction process, which requires a deep understanding of the complexity of the algorithm. One of the primary challenges in implementing the FP-Growth algorithm was handling data from a CSV file. Reading and processing the data required proper encoding detection, which was addressed using the chardet library. Another challenge was allowing the user to select specific columns for mining. This was resolved by presenting column names to the user and asking for their input to select the desired columns. To build the FP-Tree, transactions need to be processed and itemset occurrences need to be counted. The code used a dictionary to store transactions as keys and their counts as values, ensuring that duplicate transactions are combined and counted only once. This dictionary is then used to construct the FP-Tree and header table. The mining process in the FP-Growth algorithm is inherently recursive, which can lead to challenges in maintaining the state and navigating the tree. The `mine_tree()` function was implemented using a depth-first search approach, recursively mining the conditional FP-Trees and tracking the current frequent itemset using a prefix parameter. The final step in the FP-Growth algorithm is calculating the support for each frequent itemset. This was achieved by creating a separate function, `calculate_support()`, which iterates through the dataset and counts the occurrences of each frequent itemset. This process of attempting to implement the FP-growth algorithm ended up being a much bigger time investment than originally expected, which limited work done on the more important analysis and experimentation.

The biggest limitation for this project was lack of domain knowledge and prior experience. Having advice or help from a domain expert would have been an invaluable resource which could have helped inform data preprocessing decisions that could have led to much more interesting frequent pattern and association rule generation. For example, domain knowledge could have helped when deciding which columns could be removed and which ones should always be kept for mining, which would have resulted in a lot less uninteresting or unhelpful association rules. Domain knowledge would have

also been very helpful for analyzing frequent patterns and association rules to decide which ones were useful and how to change our filtering to discover even more useful patterns and associations.

Our approach to multi-level mining was also limited by the fact that by mining different versions of the same dataset, a large number of duplicate frequent patterns and association rules would be generated. The simplest way to solve this issue would be to implement automatic duplicate removal for the association rules, but time was a limiting factor in this case. Ideally, duplicate frequent patterns and association rules would be avoided by embedding the different data hierarchies into one dataset and using a modified frequent pattern mining algorithm to mine all levels of the hierarchy from a single dataset. While algorithms like this exist, they are designed for datasets where all columns in the dataset contain the same number of data hierarchy levels. Since the pipeline dataset did not have this feature, extra work would have been needed to either modify the dataset to work with existing multi-level mining algorithms, or to adapt existing multi-level mining algorithms to work with the dataset. While this problem was not accomplished in time, it would be a perfect candidate for future work.

The sheer number of association rules generated with the current method was also a limiting factor in terms of filtering out actually useful association rules. Despite many association rules being interesting based on high confidence, lift and conviction values, many of these rules did not seem particularly useful in practice. Going through and analyzing all of these association rules can be very tedious and time consuming and was therefore a limiting factor that could be potentially avoided or improved with some changes to the overall method of generating association rules.

C. Future Work

Further analysis of the generated association rules was originally planned, but due to the limitations above and the sheer amount of association rules generated, this did not fit into the time frame of this project. In the future however, this would be a worthwhile task, especially since all of the groundwork has been done and the association rules have been saved and can be easily viewed and extracted from .csv files.

Finding solutions to the limitations presented above would be the most obvious area for future work. More experimentation with data preprocessing is perhaps the most immediate and promising area of improvement. Even without the help of a domain expert, alterations to the data preprocessing have the potential to result in much more useful and interesting frequent patterns and association rules. This may also reduce the amount of useless or obvious association rules generated after frequent pattern mining.

Work on embedding the multiple data hierarchy levels into one dataset or modifying existing multilevel-mining algorithms to work with data where only some columns contain more than one data hierarchy would also be worthwhile [13]. Not only does this have the potential to improve frequent pattern mining for pipeline incident data, but the techniques and/or algorithms developed could be generalized and applied to many other datasets with spatial and temporal data. This could also help with the issue of duplicate association rules, removing the need for duplicate elimination.

Improvements or additions to the association rule generation process would also greatly benefit this project. Finding better ways to generate association rules with specific consequents or filtering out association rules based on their consequents would make it easier to discover association rules that may be useful for real world purposes such as predictions. The association rule generation could also benefit from more experimentation with minimum support, confidence, lift and conviction values in order to filter out interesting association rules.

There are a number of ways this work could be expanded upon and improved in the future. One obvious future expansion would be to apply the same process used on CER's pipeline incident dataset to other similar datasets, such as those managed by Pipeline and Hazardous Materials Safety Administration (PHMSA) for U.S. pipeline incidents [1]. Combining similar pipeline datasets to form a larger dataset may also be a good way to increase the knowledge gained from our approach and to increase confidence in our findings. By combining datasets for pipeline incidents from various countries, more interesting association rules and frequent patterns related to spatial data may also be discovered.

Expanding our approach to include cross-level mining techniques would be another natural expansion to our current work. Cross-level frequent pattern mining allows for the discovery of frequent patterns across the various levels of a data-hierarchy [13]. This would allow us to extract even more interesting association rules that could perhaps lead to even better predictions. Due to the complexity of cross-level mining however, more advanced algorithms are needed to properly mine these frequent patterns in a reasonably time efficient manner [13].

The exploration of other data hierarchies and experimentation with alternative support values also has the potential to further our work and facilitate the discovery of even more useful or interesting frequent patterns and association rules.

REFERENCES

- [1] "Advancing the safe transportation of Energy and Hazardous Materials," Pipeline and Hazardous Materials Safety Administration. [Online]. Available: <https://www.phmsa.dot.gov/>. [Accessed: 19-Apr-2023].
- [2] C. C. Aggarwal and J. Han, *Frequent Pattern Mining*. Heidelberg, NY: Springer International Publishing, 2014.
- [3] D. A. Williams and A. K. Glasmeier, "Evaluation of error across natural gas pipeline incidents," *Risk Analysis*, 2022.
- [4] G. Grahne and J. Zhu, "Fast algorithms for frequent itemset mining using FP-Trees," *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 10, pp. 1347–1362, 2005.
- [5] G. Liu, M. Boyd, M. Yu, S. Z. Halim, and N. Quddus, "Identifying causality and contributory factors of pipeline incidents by employing natural language processing and text mining techniques," *Process Safety and Environmental Protection*, vol. 152, pp. 37–46, 2021.
- [6] H. Alkharusi, "Categorical variables in regression analysis: A comparison of dummy and effect coding," *International Journal of Education*, vol. 4, no. 2, p. 202, 2012.
- [7] K. N. Aroh, I. U. Ubong, C. L. Eze, I. M. Harry, J. C. Umo - Otong, and A. E. Gobo, "Oil spill incidents and pipeline vandalization in Nigeria," *Disaster Prevention and Management: An International Journal*, vol. 19, no. 1, p. 70-87, 2010.
- [8] M. Hahsler and R. Karpienko, "Visualizing Association rules in hierarchical groups," *Journal of Business Economics*, vol. 87, no. 3, pp. 317–335, 2016.
- [9] M. Pater and D. E. Popescu, "The benefits of using prefix tree data structure in multi-level frequent pattern mining," *2007 2nd International Workshop on Soft Computing Applications*, 2007.
- [10] P. Kumari, Q. Wang, F. Khan, and J. S.-I. Kwon, "A unified causation prediction model for aboveground onshore oil and refined product pipeline incidents using artificial neural network," *Chemical Engineering Research and Design*, vol. 187, pp. 529–540, 2022.
- [11] "Pipeline Incident Data," Open Government Portal. [Online]. Available: <https://open.canada.ca/data/en/dataset/7dffdc4-23fa-440c-a36d-adf5a6cc09f1>. [Accessed: 19-Apr-2023].
- [12] S. Z. Halim, M. Yu, H. Escobar, and N. Quddus, "Towards a causal model from pipeline incident data analysis," *Process Safety and Environmental Protection*, vol. 143, pp. 348–360, Jul. 2020.
- [13] T. Eavis and X. Zheng, "Multi-level frequent pattern mining," *Database Systems for Advanced Applications*, pp. 369–383, 2009.
- [14] T. S. B. of C. Government of Canada, "Pipeline transportation occurrences in 2021," *Statistical Summary - Transportation Safety Board of Canada*, 27-Jun-2021. [Online]. Available: <http://www.tsb.gc.ca/eng/stats/pipeline/2021/ssep-sspo-2021.html>. [Accessed: 19-Apr-2023].